ACTIVIDAD TKINTER EN GIT

Instructivo del equipo. Descárgalo y síguelo completo.

CECyTEM Plantel Huixquilucan · 1000 Firmas · Un PDF de evidencia, máximo 8 MB

Lo primero, antes del programa y antes del repositorio: comprueben si Git está instalado. En la computadora y también en el teléfono.

1. Primero en la computadora

Abran PowerShell. Clic en la barra de la dirección de una carpeta, escriban powershell y pulsen Enter. Luego este comando:

```
git --version
```

Si aparece un número, por ejemplo git version 2.45.0, Git ya está. Pueden seguir.

Si dice que git no se reconoce, Git no está. Entren a https://git-scm.com/download/win y bajen Git para Windows. En el instalador pulsen Next hasta el final. En la pantalla de PATH dejen la opción Git from the command line and also from 3rd-party software.

Cierren esa ventana negra, ábranla otra vez y escriban otra vez git --version. Tiene que salir el número. Si no sale, no sigan: todavía no está instalado.

2. Primero en el teléfono

El celular no trae Git. La aplicación GitHub, la oficial, tampoco lo instala. Por eso se comprueba aparte, al inicio, antes de editar el archivo.

Android:

1. Busquen la aplicación Termux. Si no está instalada, Git no está en el teléfono.
2. Si Termux sí está, ábranla y escriban git --version.
3. Si sale un número, Git ya está. Si dice not found, no está.
4. Para instalarlo, Termux se baja de F-Droid, no de la versión vieja de Play Store. Dentro escriban pkg update, esperen, y luego pkg install git.
5. Vuelvan a escribir git --version. Sigan solo cuando salga un número, o decidan usar el navegador, que no necesita Git.

```
git --version
pkg update
pkg install git
git --version
```

iPhone:

1. Un iPhone no tiene ventana negra y no trae Git.
2. Busquen la aplicación Working Copy. Si no está instalada, Git no está en el teléfono.
3. Si Working Copy abre, Git ya viene dentro de esa aplicación. Ahí no se escribe git --version.
4. La aplicación GitHub de la App Store no instala Git. Solo sirve para ver el repositorio y aceptar la invitación.
5. Si Working Copy pide una compra para subir, no la paguen. Usen Safari. Ese camino no necesita Git, y la comprobación ya les dijo que el teléfono no lo tiene.

Si van a editar en el navegador, pueden entregar sin instalar Git en el teléfono. La comprobación se hace igual, al inicio, para no descubrirlo a la mitad.

3. Qué hace el equipo

Cada equipo hace un solo programa, distinto al de los demás. El título está en la tarjeta de SimEvaluación. Hagan ese. No elijan otro.

El programa trata de lo que dice ese título. La ventana, las cajas y los botones sirven para ese tema. Además, el equipo crea una base de datos para guardar los datos que genere la aplicación.

Todo vive en un archivo: programa.py. Ahí van tres cosas:

- La cadena de conexión a esa base de datos.
- La aplicación de Tkinter que resuelve el problema del equipo.
- El CRUD: guardar, consultar, actualizar y eliminar en esa base.

Un integrante, de preferencia el líder, crea el repositorio y sube el primer programa.py. Los demás lo clonan, cada quien hace una parte, y la suben a la misma rama: main.

4. Los programas. El de tu equipo está en la tarjeta

- 1. Tablas de multiplicar del 1 al 10. Una caja, un botón Ver tabla, y del 1 al 10 el resultado. Si el número no está entre 1 y 10, un aviso.
- 2. Conversor de Celsius a Fahrenheit. Una caja de grados Celsius y un botón Convertir. F = C × 9 / 5 + 32.
- 3. Promedio de 4 calificaciones. Cuatro cajas y un botón Promedio. Muestra el promedio con un decimal.
- 4. Lista de pendientes. Una caja, un botón Agregar y una lista. Quitar borra solo la tarea seleccionada.
- 5. Contador. Una etiqueta grande en 0. Botones +1, -1 y Reiniciar.
- 6. Metros a centímetros. Una caja de metros y un botón Convertir. Muestra metros × 100.
- 7. Ficha del alumno. Cajas de nombre, grupo y número de lista. Un botón arma la ficha.
- 8. Cuadrados del 1 al 20. Un botón lista del 1 al 20 el número al cuadrado. La última línea es 400.
- 9. Propina. Caja del total y caja del porcentaje. Muestra la propina y el total a pagar.
- 10. Precio con descuento. Caja del precio y caja del porcentaje. Avisa si el descuento no está entre 0 y 100.
- 11. Adivina el número. El programa piensa un número del 1 al 20. Dice muy alto, muy bajo o le atinaste.
- 12. Total de 3 productos. Tres nombres y tres precios. Muestra cada producto y la suma.

Esos datos del programa también se guardan en la base. El docente ve en la tarjeta cuál les tocó.

5. La base de datos y la cadena de conexión

Creen una base nueva. No reutilicen la base 302 ni la tabla test de la otra actividad. El nombre lleva el grupo y el equipo, con guion bajo, sin espacios ni acentos. Ejemplo: tkinter_301_equipo_1.

La cadena usa los mismos datos de la clase: host localhost, usuario root y contraseña vacía. Solo cambia el nombre de la base. Va al inicio de programa.py, con el comentario de quien la escribió:

```
# Ana Lopez: cadena de conexion y base de datos
import mysql.connector
import tkinter as tk
from tkinter import messagebox

def conectar():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="",
        database="tkinter_301_equipo_1",
    )
```

Antes de abrir la aplicación, la base y su tabla ya existen. MySQL tiene que estar encendido en esa computadora.

- Guardar: inserta el dato que el usuario escribió.
- Consultar: lee la tabla y la muestra en la ventana.
- Actualizar: cambia un dato que ya estaba guardado.
- Eliminar: borra un registro y, al consultar de nuevo, ya no aparece.

Después de Guardar, Actualizar o Eliminar, la ventana vuelve a consultar para que se vea el cambio.

6. Cada integrante hace algo, y deja su nombre

- Una persona: la base, la tabla y la cadena de conexión.
- Otra: la ventana de Tkinter del problema que les tocó.
- Otra: Guardar y Consultar.
- Otra: Actualizar y Eliminar.
- Si sobran personas: colores del equipo, el botón Limpiar, o el pie con los nombres de todos.

Justo arriba del cambio, en programa.py:

```
# Ana Lopez: agrego el boton Limpiar
```

El commit lleva el nombre del cambio y el nombre de esa persona:

```
git commit -m "Agrego el boton Limpiar - Ana Lopez"
```

En una computadora compartida el nombre se cambia solo dentro de la carpeta del proyecto, sin --global.

7. Qué significa cada palabra

- Guardar: Ctrl+S. El archivo queda en la carpeta de esta computadora.
- git add: preparas programa.py para el siguiente registro.
- Commit, o comprometer: el cambio queda registrado, con un mensaje y con tu nombre.
- Push, o subir: el cambio llega a GitHub, a la rama main.
- Pull, o bajar: traes lo que tus compañeros ya subieron.
- Clonar: bajas el proyecto completo la primera vez.
- Rama main: la única rama del equipo. No creen otra.

8. Revisa que Git esté a tu nombre

En cada computadora, después de comprobar que Git está instalado y antes de comprometer:

```
git config --global user.name
git config --global user.email
```

Tiene que salir tu nombre y el correo de GitHub. Si no es el tuyo, entra a la carpeta del proyecto y cámbialo solo ahí, sin --global:

```
git config user.name "Ana Lopez"
git config user.email "ana.lopez@gmail.com"
git config user.name
git config user.email
```

9. El líder crea el proyecto y sube el primer programa

1. En github.com pulsa + y New repository. El nombre es el de la tarjeta. Public. Sin Add a README.
2. En Settings, Collaborators, invita a cada integrante y al docente. Cada quien acepta en la campana.
3. Crea la carpeta y entra. Cambia el nombre por el de la tarjeta:

```
cd $env:USERPROFILE\Documents
mkdir tkinter-301-equipo-1
cd tkinter-301-equipo-1
```

4. Guarda el programa como programa.py dentro de esa carpeta, no en el escritorio.
5. Sube el primer archivo. Cambia TU-USUARIO y el mensaje por los tuyos:

```
git init
git branch -M main
git add programa.py
git commit -m "Subo el programa inicial - Ana Lopez"
git remote add origin https://github.com/TU-USUARIO/tkinter-301-equipo-1.git
git push -u origin main
```

En el push se abre el navegador. Si la ventana negra pide Password, no escribas la contraseña de GitHub. Ciérrala y entra en el navegador. Copia la dirección HTTPS y pásala al equipo.

10. Los demás clonan

Primero aceptan la invitación. Luego:

```
cd $env:USERPROFILE\Documents
git clone https://github.com/USUARIO-DEL-LIDER/tkinter-301-equipo-1.git
cd tkinter-301-equipo-1
git --version
git config user.name
```

git --version tiene que dar un número, y git config user.name tiene que decir tu nombre, antes de tocar el archivo.

11. Cada vez que hagas un cambio

Este bloque se repite. Guardar, bajar, preparar, comprometer, subir, comprobar. Entra una sola persona a la vez.

1. Guardar con Ctrl+S. Si no guardas, Git sube la versión vieja.
2. PowerShell dentro de la carpeta del repositorio.
3. git config user.name tiene que decir tu nombre.
4. Bajar lo último: git pull origin main.
5. Haz tu parte, deja el comentario con tu nombre y guarda otra vez.
6. git add programa.py y git status.
7. git commit -m "Agrego el boton Limpiar - Ana Lopez".
8. git push origin main.
9. git status debe decir working tree clean. En GitHub se ve tu nombre.

```
git pull origin main
git add programa.py
git commit -m "Agrego el boton Limpiar - Ana Lopez"
git push origin main
git status
```

12. Si algo sale mal

Si pasa esto: git --version dice que no se reconoce.

Haz esto: Git no está, o la ventana se abrió antes de instalarlo. Instálalo, cierra PowerShell, ábrelo y prueba otra vez. No sigas hasta ver el número.

Si pasa esto: En Termux, git --version dice not found.
Haz esto: En el teléfono Android, Git no está. Escribe pkg update y luego pkg install git. Vuelve a probar git --version.

Si pasa esto: En el iPhone no hay dónde escribir git --version.
Haz esto: Es normal. Busca Working Copy. Si no está, Git no está en el iPhone. Para entregar, usa Safari en github.com.

Si pasa esto: El commit en GitHub está a nombre de otra persona.
Haz esto: Entra a la carpeta del proyecto y pon tu nombre sin --global. El commit viejo se queda. El siguiente tiene que salir con el tuyo.

Si pasa esto: La ventana negra pide Password.
Haz esto: No escribas la contraseña de GitHub. Ciérrala y entra en el navegador.

Si pasa esto: Dice rejected o failed to push.
Haz esto: Alguien subió antes. Baja y vuelve a subir. Si se abre un editor, pulsa Esc, escribe :wq y pulsa Enter.

```
git pull origin main --no-edit
git push origin main
```

Si pasa esto: nothing to commit, pero tu cambio no está.
Haz esto: No guardaste, o editaste una copia del escritorio. Abre el programa.py de la carpeta del repositorio y guarda con Ctrl+S.

Si pasa esto: fatal: not a git repository.
Haz esto: No estás dentro de la carpeta. cd $env:USERPROFILE\Documents y luego cd y el nombre del repositorio.

Si pasa esto: remote origin already exists.
Haz esto: No la agregues otra vez. Si la dirección está mal:

```
git remote set-url origin https://github.com/TU-USUARIO/tkinter-301-equipo-1.git
```

Si pasa esto: 403 o Permission denied.
Haz esto: Acepta la invitación en la campana. El líder te agrega en Collaborators.

Si pasa esto: Aparecen <<<<<<< en programa.py.
Haz esto: Dos personas editaron lo mismo. Quita esas marcas, deja los dos cambios y compromete: Junto los dos cambios - Tu Nombre.

Si pasa esto: La aplicación no abre o no encuentra mysql.connector.
Haz esto: Se prueba en la computadora: pip install mysql-connector-python. El celular no abre Tkinter.

Si pasa esto: No puede conectar o la base no existe.
Haz esto: host localhost, user root, password vacío, y el database con el nombre que sí crearon. No usen la base 302.

Si pasa esto: Aviso LF will be replaced by CRLF.
Haz esto: No es un error. Sigue.

13. Si el teléfono no tiene Git y van a editar ahí

Ya lo comprobaron al inicio. Si git no está, no lo necesitan para entregar. La aplicación GitHub, la oficial, sirve para aceptar la invitación. El cambio se hace en el navegador, en https://github.com.

En el celular, el botón Commit changes guarda, compromete y sube en un solo paso.

Android: Chrome, el repositorio, programa.py, el lápiz, el comentario con tu nombre, y Commit changes en la rama main. Si no aparece el lápiz, elige Sitio de escritorio.

iPhone: Safari, el botón aA, Sitio de escritorio, el mismo camino. Working Copy solo hace falta si quieren los

comandos de la computadora. Si pide compra para subir, usen Safari.

No entreguen el programa desde un editor de Python del celular. Eso no llega a GitHub.

14. La evidencia que sí se sube a SimEvaluación

Es otro PDF, uno por equipo, de 8 MB o menos. Este instructivo no cuenta como evidencia.

- Portada: de qué trata el programa, el equipo, los integrantes y el enlace del repositorio.
- La base de datos, la cadena de conexión y el CRUD funcionando en Tkinter.
- El comentario de cada integrante dentro de programa.py.

15. Orden final

1. Comprueben Git en la computadora con git --version.
2. Si van a usar el teléfono, comprueben también ahí: Termux en Android, Working Copy en iPhone.
3. Vean en la tarjeta cuál programa les tocó.
4. El líder sube programa.py. Cada quien clona, guarda, compromete con su nombre y sube a main.
5. Si sale un error, busquen la frase en la sección 12.
6. Suban el PDF de evidencia. No suban este instructivo.